



A

Ansible

Complete Guide

Urdu/Hindi mein — Step by Step

- Installation & Setup
- Inventory (Static & Dynamic)
- Playbooks, Variables & Roles
- AWS Dynamic Inventory
- Production Deployment
- Ansible Vault & Secrets
- 100+ Real Code Examples

Table of Contents

- 1 Ansible Kya Hai?**
Architecture, concepts, kab use karo
- 2 Installation & Setup**
Ubuntu, RHEL, macOS, SSH keys, ansible.cfg
- 3 Inventory**
Static INI/YAML, groups, host_vars/group_vars
- 4 Playbooks**
Tasks, handlers, loops, conditions, templates
- 5 Variables & Roles**
Priority, role structure, Galaxy
- 6 AWS Dynamic Inventory**
boto3, aws_ec2.yml, tags, multi-env
- 7 Production Deployment**
Rolling updates, CI/CD, rollback
- 8 Ansible Vault**
Secrets encrypt karna, vault use karna
- 9 Zaroori Modules**
Files, packages, users, AWS modules, debug

1

Ansible Kya Hai?

Architecture, Core Concepts, Kab Use Karo

Ansible ek open-source IT automation tool hai jo aapko servers configure karne, applications deploy karne aur infrastructure manage karne deta hai — bina kisi agent ke install kiye. Aap ek YAML file (Playbook) likhte hain jisme batate hain 'kya karna hai' — Ansible SSH se connect karke sab apne aap kar deta hai.

i Core Idea — Agentless

Managed nodes pe koi extra software install karne ki zaroorat nahin. Sirf SSH aur Python (2.7+ ya 3.5+) kaafi hai. Control node se SSH se connect karke kaam karta hai.

Ansible ke 6 Core Concepts

Control Node	Aapka machine jahan Ansible install hai. Jahan se sab commands chalte hain.
Managed Nodes	Target servers jahan kaam hoga. SSH + Python kaafi hai, koi agent nahin.
Inventory	Server list ki file. Groups define karte hain. Static (INI/YAML) ya Dynamic (AWS/GCP).
Playbook	YAML file jisme tasks likhe hain. Kya karna hai, kaun se server pe, kaise.
Modules	Ansible ke built-in tools (3000+). apt, yum, copy, service, user, git, etc.
Roles	Reusable task packages. Ek role = ek kaam (nginx, mysql, docker).

Ansible Kab Use Karo?

Use Case	Example
Server Configuration	100 servers pe nginx install, users create karo
Application Deployment	Git se latest code pull karo, service restart karo
Cloud Provisioning	AWS EC2, S3, RDS create/manage karo
Security Hardening	SSH config, firewall rules, patches apply karo
CI/CD Pipeline	Jenkins/GitHub Actions se trigger hoke deploy karo

+ Idempotent Kya Hota Hai?

Ansible bar bar chalao, wahi result milega. Agar nginx already installed hai toh dobara install nahin karega. Ye production mein bahut important hai — safe rerun!

2

Installation & Setup

Ubuntu, RHEL, macOS, SSH Keys, ansible.cfg

Ubuntu / Debian pe Install

Terminal

```
# Ubuntu/Debian pe install
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository --yes --update ppa:ansible/ansible
sudo apt install -y ansible

# Version check
ansible --version
```

RHEL / CentOS / Amazon Linux

Terminal

```
# RHEL/CentOS pe install
sudo yum install -y epel-release
sudo yum install -y ansible

# RHEL 8+ / Amazon Linux 2023
sudo dnf install -y ansible

# AWS collections bhi install karo
ansible-galaxy collection install amazon.aws
pip install boto3 botocore
```

SSH Key Setup — Managed Nodes ke liye

Step 1

Control node pe SSH key generate karo: `ssh-keygen -t ed25519 -C "ansible-key"`

Step 2

Public key servers pe copy karo: `ssh-copy-id -i ~/.ssh/id_ed25519.pub user@192.168.1.10`

Step 3

Test karo: `ansible all -i "192.168.1.10," -m ping --user ubuntu`

ansible.cfg — Global Configuration

ansible.cfg

```
[defaults]
inventory = ./inventory
remote_user = ubuntu
private_key_file = ~/.ssh/id_ed25519
host_key_checking = False
retry_files_enabled = False
stdout_callback = yaml

[privilege_escalation]
become = True
become_method = sudo
become_user = root
```

3

Inventory — Server List

Static INI/YAML, Groups, Variables

Inventory batata hai Ansible ko kaun se servers hain, unke groups kya hain, aur unse connect kaise karein.

hosts.ini — Static Inventory (INI Format)

```
inventory/hosts.ini

# Single hosts
mail.example.com

# Web servers group
[webservers]
web1.example.com
web2.example.com ansible_port=2222
192.168.1.10      ansible_user=admin

# DB servers group
[dbservers]
db1.example.com ansible_user=ubuntu
db2.example.com

# Range se servers (web01 se web10 tak)
[appservers]
app[01:10].example.com

# Groups of Groups
[production:children]
webservers
dbservers

[production:vars]
env=production
ansible_user=deploy
```

inventory.yml — YAML Format

```
inventory/inventory.yml

all:
  vars:
    ansible_user: ubuntu
    ansible_ssh_private_key_file: ~/.ssh/mykey.pem
  children:
    webservers:
      hosts:
        web1:
          ansible_host: 10.0.1.10
          http_port: 80
        web2:
          ansible_host: 10.0.1.11
    dbservers:
      hosts:
        db-primary:
          ansible_host: 10.0.2.10
          db_role: primary
```

host_vars aur group_vars — Directory Structure

```
Directory Tree

inventory/
├── hosts.ini
├── group_vars/
│   ├── all.yml           # Sab servers ke variables
│   ├── webservers.yml    # Sirf web servers ke variables
│   ├── dbservers.yml     # Sirf DB servers ke variables
│   └── host_vars/
│       ├── web1.yml      # Sirf web1 ke variables
│       └── db1.yml       # Sirf db1 ke variables
```

```
group_vars/webservers.yml

---
nginx_port: 80
app_env: production
max_workers: 4

# group_vars/all.yml
---
ansible_user: ubuntu
ntp_server: time.google.com
```

Ad-hoc Commands — Bina Playbook ke

Terminal

```
# Sab servers ping karo
ansible all -i hosts.ini -m ping

# Webservers pe uptime check
ansible webservers -i hosts.ini -m command -a 'uptime'

# Package install karo
ansible webservers -i hosts.ini -m apt -a 'name=nginx state=present' --become

# Service restart karo
ansible webservers -i hosts.ini -m service -a 'name=nginx state=restarted' --become

# Disk space check
ansible all -i hosts.ini -m shell -a 'df -h'
```


4

Playbooks — YAML Automation

Tasks, Handlers, Loops, Conditions, Templates

Playbook wo jagah hai jahan aap likhte hain kya karna hai — kis server pe, kya tasks, kya conditions. YAML format mein hota hai.

Playbook ki Anatomy — Pura Example

deploy-nginx.yml

```
---
- name: Nginx install aur configure karo
  hosts: webservers
  become: yes
  vars:
    nginx_port: 80
    server_name: myapp.example.com

  tasks:
    - name: Nginx install karo
      apt:
        name: nginx
        state: present
        update_cache: yes

    - name: Config file copy karo
      template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/nginx.conf
        mode: '0644'
      notify: Nginx restart karo

    - name: Nginx service enable karo
      service:
        name: nginx
        state: started
        enabled: yes

  handlers:
    - name: Nginx restart karo
      service:
        name: nginx
        state: restarted
```

Loops — Multiple Items pe Kaam Karo

loops-example.yml

```
# Loop: Multiple packages install karo
- name: Packages install karo
  apt:
    name: '{{ item }}'
    state: present
  loop:
    - git
    - curl
    - htop
    - vim

# Loop: Dictionary ke sath users create
- name: Users create karo
  user:
    name: '{{ item.name }}'
    groups: '{{ item.groups }}'
    state: present
  loop:
    - { name: ali,      groups: sudo }
    - { name: ahmed,   groups: docker }
    - { name: fatima,  groups: deploy }
```

Conditions aur Register

conditions.yml

```
# Condition: Sirf Ubuntu pe run karo
- name: Ubuntu pe apt update
  apt:
    update_cache: yes
  when: ansible_os_family == 'Debian'

# Register: Task ka output save karo
- name: Git clone karo
  git:
    repo: https://github.com/myorg/myapp.git
    dest: /var/www/myapp
    register: git_result

- name: Deploy hone pe restart karo
  service:
    name: myapp
    state: restarted
  when: git_result.changed
```

Jinja2 Templates

templates/nginx.conf.j2

```
server {
    listen {{ nginx_port }};
    server_name {{ server_name }};

    location / {
        proxy_pass http://{{ backend_host }}:{{ backend_port }};
    }

    {% if ssl_enabled %}
    listen 443 ssl;
    ssl_certificate {{ ssl_cert_path }};
    {% endif %}

    {% for domain in extra_domains %}
    server_name {{ domain }};
    {% endfor %}
}
```

Playbook Chalane ke Commands

Terminal

```
# Normal run
ansible-playbook -i inventory/hosts.ini deploy.yml

# Dry run (kuch change nahin karega)
ansible-playbook deploy.yml --check

# Verbose output (debug)
ansible-playbook deploy.yml -vvv

# Sirf specific hosts pe
ansible-playbook deploy.yml --limit web1.example.com

# Extra variables
ansible-playbook deploy.yml --extra-vars 'version=2.0 env=staging'

# Sirf deploy tag wale tasks
ansible-playbook deploy.yml --tags deploy

# List tasks without running
ansible-playbook deploy.yml --list-tasks
```

5

Variables & Roles

Priority Order, Role Structure, Ansible Galaxy

Variable Priority Order (Neeche wala Override karta hai)

Priority	Jagah	Example
1 - Lowest	Role defaults	roles/nginx/defaults/main.yml
2	group_vars/all	inventory/group_vars/all.yml
3	group_vars/groupname	inventory/group_vars/webserver.yml
4	host_vars/hostname	inventory/host_vars/web1.yml
5	Playbook vars	vars: section in playbook
6	vars_files	vars_files: [secrets.yml]
7 - Highest	Extra vars (CLI)	--extra-vars 'key=value'

Roles — Directory Structure

Terminal — Role Create karo

```
# Role create karo
ansible-galaxy init roles/nginx

# Structure:
roles/
├── nginx/
│   ├── defaults/main.yml # Default variables
│   ├── vars/main.yml     # Role-specific variables
│   ├── tasks/main.yml    # Main tasks
│   ├── handlers/main.yml # Handlers
│   ├── templates/        # Jinja2 templates
│   ├── files/            # Static files
│   ├── meta/main.yml     # Dependencies
│   └── README.md

```

roles/nginx/tasks/main.yml

```
---
- name: Nginx install karo
  apt:
    name: nginx
    state: present
    update_cache: yes

- name: Config deploy karo
  template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: nginx restart

- name: Service enable karo
  service:
    name: nginx
    state: started
    enabled: yes
```

site.yml — Master Playbook

```
---
- name: Web servers configure karo
  hosts: webservers
  become: yes
  roles:
    - common      # Base setup
    - nginx       # Web server
    - app         # Application

- name: DB servers configure karo
  hosts: dbservers
  become: yes
  roles:
    - common
    - mysql

# Role variables override karna:
- name: Staging web servers
  hosts: staging_web
  roles:
    - role: nginx
      vars:
        nginx_port: 8080
```

Ansible Galaxy — Community Roles

```
# Community se roles download karo
ansible-galaxy install geerlingguy.nginx
ansible-galaxy install geerlingguy.mysql
ansible-galaxy install geerlingguy.docker

# requirements.yml se sab install karo:
---
roles:
  - name: geerlingguy.nginx
    version: '3.1.0'
  - name: geerlingguy.mysql
    version: '4.3.0'

# Phir install:
ansible-galaxy install -r requirements.yml
```

6

AWS Dynamic Inventory

boto3, aws_ec2.yml, EC2 Tags, Multi-Environment

Static inventory mein bar bar IPs update karna padta hai. AWS Dynamic Inventory automatically EC2 instances fetch karta hai — start hote hi Ansible unhe detect kar leta hai.

+ Kyu Zaroori Hai?

Aap 50 naye servers start karo, kuch nahin likhna. Ansible automatically unhe groups mein add kar leta hai by tag, region, environment. Auto-scaling ke saath perfect kaam karta hai.

Step 1: Install boto3 aur AWS Collection

Terminal

```
# Python packages install karo
pip install boto3 botocore

# Ansible AWS collection install karo
ansible-galaxy collection install amazon.aws
ansible-galaxy collection install community.aws

# Verify
ansible-galaxy collection list | grep amazon
```

Step 2: AWS Credentials Setup

~/.aws/credentials

```
# Option 1: Credentials file (recommended)
[default]
aws_access_key_id = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/K7MDENG...
region = ap-south-1

# Option 2: Environment variables
export AWS_ACCESS_KEY_ID='AKIAIOSFODNN7EXAMPLE'
export AWS_SECRET_ACCESS_KEY='wJalrXUtn...'
export AWS_DEFAULT_REGION='ap-south-1'

# Option 3: IAM Role (EC2 pe best practice)
# Control node ko IAM Role assign karo
# EC2ReadOnly permissions kaafi hain
```

Step 3: aws_ec2.yml — Main Dynamic Inventory File

inventory/aws_ec2.yml

```
---
plugin: amazon.aws.aws_ec2

# AWS Regions
regions:
  - ap-south-1      # Mumbai
  - us-east-1       # N. Virginia

# Sirf running instances fetch karo
filters:
  instance-state-name: running
  'tag:ManagedBy': ansible

# EC2 instances ko groups mein assign karo
keyed_groups:
  # Tag 'Environment' se group
  - key: tags.Environment
    prefix: env
    separator: '_'

  # Tag 'Role' se group
  - key: tags.Role
    prefix: role
    separator: '_'

  # Region se group
  - key: placement.region
    prefix: aws_region

# Hostname kya use kare
hostnames:
  - private-ip-address

# Compose variables
compose:
  ansible_host: private_ip_address
  ansible_user: "ubuntu"
  aws_instance_id: instance_id
```

Step 4: EC2 Tags Setup

! Important!

AWS Console ya Terraform se ye tags lagao: Environment: production | Role: webserver | Project: myapp | ManagedBy: ansible. Inhi tags se Ansible automatic groups banata hai.

Terraform — EC2 Tags

```
resource "aws_instance" "web" {
  ami          = "ami-0f5ee92e2d63afc18"
  instance_type = "t3.micro"

  tags = {
    Name          = "web-server-1"
    Environment   = "production"
    Role          = "webserver"
    Project       = "myapp"
    ManagedBy     = "ansible"
  }
}
```

AWS CLI — Tags add karo

```
aws ec2 create-tags \
  --resources i-1234567890abcdef0 \
  --tags Key=Environment,Value=production \
        Key=Role,Value=webserver \
        Key=ManagedBy,Value=ansible
```

Step 5: Test aur Use karo

Terminal

```
# Inventory list dekho
ansible-inventory -i inventory/aws_ec2.yml --list

# Graph format mein
ansible-inventory -i inventory/aws_ec2.yml --graph

# Output example:
# @all:
#   |--@env_production:
#   |   |--10.0.1.10
#   |   |--10.0.1.11
#   |--@role_webserver:
#   |   |--10.0.1.10
#   |--@role_dbserver:
#   |   |--10.0.2.20

# Sab production servers ping karo
ansible env_production -i inventory/aws_ec2.yml -m ping

# Sirf webserver pe playbook
ansible-playbook -i inventory/aws_ec2.yml deploy.yml --limit role_webserver
```

Multi-Environment group_vars Setup

group_vars/env_production.yml

```
---
ansible_user: ubuntu
ansible_ssh_private_key_file: ~/.ssh/prod.pem
app_env: production
db_host: prod-db.internal
app_replicas: 3

# group_vars/env_staging.yml
---
ansible_user: ubuntu
ansible_ssh_private_key_file: ~/.ssh/staging.pem
app_env: staging
db_host: staging-db.internal
app_replicas: 1
```

7

Production Deployment

Rolling Updates, CI/CD, Rollback Strategy

Production mein Ansible sirf ek playbook chalana nahin hai. Proper structure, CI/CD integration, error handling, rolling updates — sab dekhte hain.

Production Directory Structure

Project Tree

```
myproject/
├── ansible.cfg
├── site.yml           # Master playbook
├── deploy.yml         # Deploy playbook
├── requirements.yml   # Galaxy requirements
├──
├── inventory/
│   ├── aws_ec2.yml    # Dynamic inventory
│   ├── group_vars/
│   │   ├── all.yml
│   │   ├── env_production.yml
│   │   └── env_staging.yml
│   └── host_vars/
│       └──
├── roles/
│   ├── common/        # Base setup
│   ├── nginx/         # Web server
│   ├── app/           # Application
│   └── monitoring/     # Prometheus/Grafana
│       └──
├── vars/
│   ├── main.yml
│   └── secrets.yml     # Ansible Vault encrypted
│       └──
├── .github/
│   └── workflows/
│       └── deploy.yml  # GitHub Actions CI/CD
```

Rolling Update — Zero Downtime Deployment

rolling-deploy.yml

```
---
- name: Rolling deployment – zero downtime
  hosts: role_webserver
  become: yes
  serial: 2          # Ek baar mein sirf 2 servers
  max_fail_percentage: 25

  pre_tasks:
    - name: Load balancer se server remove
      uri:
        url: http://lb.internal/api/disable/{{ inventory_hostname }}
        method: POST
      delegate_to: lb.internal

    - name: Connections khatam hone do
      pause:
        seconds: 30

  tasks:
    - name: Naya code deploy karo
      git:
        repo: https://github.com/org/app.git
        dest: /var/www/app
        version: '{{ deploy_version | default('main') }}'

    - name: Health check karo
      uri:
        url: http://{{ ansible_host }}:8000/health
        status_code: 200
      retries: 5
      delay: 10

  post_tasks:
    - name: LB mein wapas add karo
      uri:
        url: http://lb.internal/api/enable/{{ inventory_hostname }}
        method: POST
      delegate_to: lb.internal
```

CI/CD — GitHub Actions ke sath

.github/workflows/deploy.yml

```
name: Production Deploy
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Ansible install
        run: |
          pip install ansible boto3 botocore
          ansible-galaxy collection install amazon.aws

      - name: SSH key setup
        run: |
          mkdir -p ~/.ssh
          echo '${{ secrets.SSH_PRIVATE_KEY }}' > ~/.ssh/deploy.pem
          chmod 600 ~/.ssh/deploy.pem

      - name: Deploy karo
        env:
          AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
          AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
          AWS_DEFAULT_REGION: ap-south-1
        run: |
          ansible-playbook rolling-deploy.yml \
            --vault-password-file .vault_pass \
            --extra-vars 'deploy_version=${{ github.sha }}'
```

Rollback Strategy

deploy-with-rollback.yml

```
---
- name: Deploy with rollback support
  hosts: webservers
  become: yes
  vars:
    release_path: /var/www/releases/{{ ansible_date_time.epoch }}
    current_path: /var/www/current

  tasks:
    - name: Release directory banao
      file:
        path: '{{ release_path }}'
        state: directory

    - name: Code deploy karo
      git:
        repo: https://github.com/org/app.git
        dest: '{{ release_path }}'
        register: git_deploy
        ignore_errors: yes

    - name: Rollback agar deploy fail
      file:
        path: '{{ release_path }}'
        state: absent
      when: git_deploy.failed

    # Atomic symlink swap
    - name: Current symlink update karo
      file:
        src: '{{ release_path }}'
        dest: '{{ current_path }}'
        state: link
        force: yes

    # Purani releases cleanup
    - name: Old releases delete karo
      shell: ls -dt /var/www/releases/* | tail -n +6 | xargs rm -rf
      ignore_errors: yes
```

8

Ansible Vault — Secrets Secure Karna

Encrypt, Decrypt, CI/CD Integration



Security Rule

Kabhi bhi plaintext passwords ya API keys Git mein commit mat karo. Database passwords, API keys, SSH keys — sab Vault mein encrypt karo.

Basic Vault Commands

Terminal — Vault Commands

```
# Naya encrypted file banao
ansible-vault create vars/secrets.yml

# Existing file encrypt karo
ansible-vault encrypt vars/secrets.yml

# Encrypted file edit karo
ansible-vault edit vars/secrets.yml

# Decrypt karke dekho (careful!)
ansible-vault view vars/secrets.yml

# Decrypt karo (permanent)
ansible-vault decrypt vars/secrets.yml

# Password change karo
ansible-vault rekey vars/secrets.yml
```

vars/secrets.yml (before encryption)

```
---
db_password: 'MyStr0ngPassw0rd!'
api_key: 'sk-xxxxxxxxxxxxxxxxxxxxxxxxxx'
aws_secret: 'wJalrXUtnFEMI/K7MDENG...'
jwt_secret: 'super-secret-jwt-key'
smtp_password: 'emailpass123'
```

Playbook mein Vault Use karo

playbook.yml

```
---
- name: App configure karo
  hosts: webservers
  vars_files:
    - vars/main.yml
    - vars/secrets.yml      # Encrypted file

  tasks:
    - name: DB config deploy karo
      template:
        src: templates/database.conf.j2
        dest: /etc/app/database.conf
        # Template mein {{ db_password }} use hoga

# Chalane ke tarike:

# 1. Password manually type karo
ansible-playbook playbook.yml --ask-vault-pass

# 2. Password file se (CI/CD ke liye)
echo 'myVaultPassword' > .vault_pass
ansible-playbook playbook.yml --vault-password-file .vault_pass
```

Single Variable Encrypt karo

Terminal

```
# Sirf ek variable encrypt karo
ansible-vault encrypt_string 'MyStr0ngPassw0rd!' --name 'db_password'

# group_vars/all.yml mein aise use karo:
app_port: 8080           # Plain text
app_name: myapp          # Plain text
db_password: !vault |    # Encrypted
                        $ANSIBLE_VAULT;1.1;AES256
                        38396635623464366633...
```


9

Zaroori Ansible Modules

Files, Packages, Services, Users, AWS, Debug

Ansible mein 3000+ modules hain. Yahan production mein sabse zyada use hone wale modules ke examples hain.

File Modules

```
file-modules.yml

# copy — local file ko remote pe copy
- name: Config copy karo
  copy:
    src: files/nginx.conf
    dest: /etc/nginx/nginx.conf
    owner: root
    mode: '0644'
    backup: yes

# file — directory create ya delete
- name: Directory banao
  file:
    path: /var/www/app
    state: directory
    owner: www-data
    mode: '0755'
    recurse: yes

# lineinfile — file mein specific line add
- name: SSH config update
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^PasswordAuthentication'
    line: 'PasswordAuthentication no'
    validate: sshd -t -f %s

# fetch — remote file download karo
- name: Log file download
  fetch:
    src: /var/log/app.log
    dest: ./logs/{{ inventory_hostname }}_app.log
    flat: yes
```

Package & Service Modules

packages-services.yml

```
# apt - Ubuntu packages
- name: Packages install karo
  apt:
    name:
      - nginx
      - python3
      - git
    state: present
    update_cache: yes

# pip - Python packages
- name: Python packages install
  pip:
    name: [django, gunicorn, redis]
    virtualenv: /var/www/venv

# service - Services manage karo
- name: Nginx start karo
  service:
    name: nginx
    state: started
    enabled: yes

# systemd - Custom service
- name: Systemd reload karo
  systemd:
    daemon_reload: yes
    name: myapp
    state: started
    enabled: yes
```

User & Network Modules

users-network.yml

```
# user — Users manage karo
- name: Deploy user banao
  user:
    name: deploy
    groups: sudo,docker
    shell: /bin/bash
    state: present

# authorized_key — SSH key add karo
- name: SSH key add karo
  authorized_key:
    user: deploy
    key: "{{ lookup('file', '~/.ssh/id_ed25519.pub') }}"
    state: present

# cron — Cron jobs manage karo
- name: Backup cron job
  cron:
    name: 'Daily DB backup'
    minute: '0'
    hour: '2'
    job: /usr/local/bin/backup.sh

# uri — HTTP requests (health checks)
- name: Health check
  uri:
    url: http://localhost:8000/health
    status_code: 200
    retries: 5
    delay: 10
```

AWS Modules — Cloud Provisioning

aws-modules.yml

```
# EC2 instance create karo
- name: EC2 instance launch karo
  amazon.aws.ec2_instance:
    name: 'web-server-{{ env }}'
    instance_type: t3.micro
    image_id: ami-0f5ee92e2d63afc18
    region: ap-south-1
    key_name: my-keypair
    security_groups: [web-sg]
    tags:
      Environment: '{{ env }}'
      Role: webserver
      ManagedBy: ansible
    state: running

# S3 bucket banao
- name: S3 bucket create karo
  amazon.aws.s3_bucket:
    name: myapp-assets-{{ env }}
    region: ap-south-1
    versioning: yes
    state: present

# RDS instance
- name: RDS database create karo
  community.aws.rds_instance:
    db_instance_identifier: myapp-db
    db_instance_class: db.t3.micro
    engine: mysql
    master_username: admin
    master_user_password: '{{ db_password }}'
    state: present
```

Debug Modules

debug-modules.yml

```

# debug — Variables print karo
- name: Variable check karo
  debug:
    var: ansible_os_family

- name: Custom message
  debug:
    msg: 'Server {{ inventory_hostname }} IP: {{ ansible_host }}'

# assert — Condition check karo
- name: Python version check
  assert:
    that:
      - ansible_python_version is version('3.8', '>=')
    fail_msg: 'Python 3.8+ chahiye!'

# Common facts:
# ansible_hostname, ansible_os_family
# ansible_processor_vcpus, ansible_memtotal_mb
# ansible_default_ipv4.address
# ansible_distribution_version

```

Quick Reference — Common Commands

Task	Command
Ping all servers	ansible all -m ping
Run playbook	ansible-playbook deploy.yml
Dry run	ansible-playbook deploy.yml --check
Specific tags	ansible-playbook deploy.yml --tags deploy
Dynamic inventory list	ansible-inventory -i aws_ec2.yml --list
Vault encrypt	ansible-vault encrypt secrets.yml
Install role	ansible-galaxy install geerlingguy.nginx
Debug verbose	ansible-playbook deploy.yml -vvv

Ansible — Infrastructure as Code

Ye guide aapko Ansible ki shuruaat se production tak le jaati hai.